

Trường Đại Học Ngoại Ngữ - Tin Học Thành Phố Hồ Chí Minh

Khoa Công Nghệ Thông Tin



BÁO CÁO KẾT THÚC MÔN HỌC ĐIỆN TOÁN ĐÁM MÂY

ĐỀ TÀI: WEBSITE XEM THỜI TIẾT THEO THÀNH PHỐ

Giảng viên Hướng Dẫn : Ths Cao Tiến Thành
Nhóm 5

Thành Viên :

1. Huỳnh Hữu Nghiệp – MSSV: 21DH113908
2. Lê Trần Bảo Khang – MSSV: 23DH111490
3. Bùi Thủy Ngọc Duyên – MSSV: 22DH483978

Tp. Hồ Chí Minh, tháng 07 năm 2026

Nhận xét của giảng viên hướng dẫn

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

TP. HCM, tháng...07...năm 2026

Giảng viên hướng dẫn

(Ký tên, đóng dấu)

MỤC LỤC

TÓM TẮT ĐỀ TÀI	8
CHƯƠNG I: TỔNG QUAN	9
1.1. Đặt vấn đề	9
1.2. Động lực nghiên cứu	10
1.3. Phạm vi đề tài	10
1.4. Đối tượng nghiên cứu và triển khai	11
1.5. Kết quả dự kiến	12
CHƯƠNG II: CƠ SỞ LÝ THUYẾT	13
2.1. Các định nghĩa và khái niệm cơ bản	13
2.1.1. Kiến trúc không máy chủ (Serverless Architecture)	13
2.1.2. Mô hình FaaS (Function-as-a-Service)	14
2.1.3. Giao diện lập trình ứng dụng (API) và API Key	15
2.1.4. Nền tảng điện toán đám mây AWS Amplify	15
2.1.5. Nền tảng Firebase (Backend-as-a-Service)	16
2.2. Phân loại và so sánh kỹ thuật	17
2.2.1. So sánh kiến trúc máy chủ Truyền thống (Monolithic/VPS) và kiến trúc không máy chủ (ServerLess)	17
2.2.2. Phân tích ưu điểm của giải pháp AWS Amplify & Firebase	19
2.2.3 Nguyên lý luồng dữ liệu bảo mật trong kiến trúc ứng dụng	19
CHƯƠNG III: PHƯƠNG PHÁP THỰC HIỆN.....	22
3.1. Lựa chọn phương pháp tiếp cận	22
3.2. Thiết kế hệ thống và luồng xử lý	23
3.3. Sơ đồ kiến trúc mạng	24
CHƯƠNG IV: THỰC NGHIỆM VÀ ĐÁNH GIÁ	26
4.1. Triển khai và Demo sản phẩm	26
4.1.1. Môi trường triển khai thực tế	26
4.1.2. Giao diện và Các tính năng cốt lõi	27
4.2. Đánh giá kết quả (Evaluation).....	31
4.2.1. Ưu điểm	31
4.2.2. Hạn chế và Hướng khắc phục	32
CHƯƠNG V: KẾT LUẬN	33
5.1. Kết quả đạt được	33
5.2. Hướng phát triển trong tương lai	34

5.3. Bảng phân công công việc.....	35
5.4. Tài liệu tham khảo.....	36

DANH MỤC HÌNH ẢNH

Hình 1 : ServerLess Architecture	13
Hình 2 : Faas.....	14
Hình 3 : AWS Amplify.....	16
Hình 4 : Firebase	17
Hình 5 : Minh họa kiến trúc dòng chảy dữ liệu hệ thống	20
Hình 6 : Sơ đồ thiết kế hệ thống và luồng xử lý.....	23
Hình 7 : Sơ đồ kiến trúc mạng.....	24
Hình 8 : Giao diện quản lý dự án (Deploy thành công lên đám mây)	26
Hình 9 : biến môi trường (Environment variables) trên AWS	27
Hình 10 : Giao diện chính của ứng dụng với bản đồ tương tác và bảng tra cứu thời tiết chi tiết.....	28
Hình 11 : Dữ liệu lịch sử tra cứu được lưu trữ và đồng bộ thời gian thực trên Firebase Firestore	29
Hình 12 : Giao diện ứng dụng tự động co giãn và tối ưu hiển thị trên thiết bị di động.....	30
Hình 13 : Các API KEY được cất giấu trong .env.local.....	31
Hình 14 : Kiểm tra luồng gọi API qua công cụ Developer Tools cho thấy khóa bí mật đã được mã hóa an toàn	32

DANH MỤC BẢNG

Bảng 1 : Bảng danh mục từ viết tắt	7
Bảng 2 : So sánh các đặt tính kỹ thuật giữa kiến trúc VPS truyền thống và Serverless.....	19
Bảng 3 : Bảng phân công công việc	36

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Giải thích Tiếng Anh	Giải thích Tiếng Việt
API	Application Programming Interface	Giao diện lập trình ứng dụng
AWS	Amazon Web Services	Dịch vụ điện toán đám mây của Amazon
CDN	Content Delivery Network	Mạng phân phối nội dung
CI/CD	Continuous Integration / Continuous Deployment	Tích hợp liên tục / Triển khai tự động liên tục
CLI	Command Line Interface	Giao diện dòng lệnh
DdoS	Distributed Denial of Service	Tấn công từ chối dịch vụ phân tán
FaaS	Function-as-a-Service	Mô hình tính toán dạng hàm
GCP	Google Cloud Platform	Nền tảng điện toán đám mây của Google
IAM	Identity and Access Management	Quản lý quyền truy cập và định danh
ISCM	Information Security Continuous Monitoring	Giám sát an toàn thông tin liên tục

Bảng 1 : Bảng danh mục từ viết tắt

TÓM TẮT ĐỀ TÀI

Đề tài "Website xem thời tiết theo thành phố" được thực hiện nhằm giải quyết thách thức về chi phí vận hành, khả năng chịu tải và nguy cơ lộ mật khẩu API trên các hệ thống web truyền thống.

Mục tiêu của đề tài là xây dựng một trang web tra cứu thời tiết có giao diện tự động thích ứng (Responsive) và tốc độ phản hồi nhanh. Về phương pháp thực hiện, hệ thống áp dụng kiến trúc Serverless và Backend-as-a-Service (BaaS). Trong đó, ứng dụng được triển khai trên hạ tầng đám mây của AWS Amplify để tối ưu CI/CD và bảo mật API Key thông qua biến môi trường.

Đồng thời, hệ thống tích hợp nền tảng Firebase để quản lý định danh người dùng và đồng bộ hóa cơ sở dữ liệu lịch sử thời gian thực. Sản phẩm thực nghiệm là một website hoàn chỉnh, vận hành ổn định trên Internet với chi phí duy trì tối thiểu và độ bảo mật cao

CHƯƠNG I: TỔNG QUAN

1.1. Đặt vấn đề

Trong kỷ nguyên số hóa và biến đổi khí hậu toàn cầu như hiện nay, thông tin thời tiết thời gian thực (real-time) đã trở thành một yếu tố thiết yếu, ảnh hưởng trực tiếp đến chuỗi cung ứng logistics, hoạt động sản xuất và đời sống sinh hoạt hằng ngày của con người. Sự phát triển của công nghệ web đã mở ra khả năng tiếp cận thông tin này một cách nhanh chóng thông qua việc tích hợp các giao diện lập trình ứng dụng (API) từ các tổ chức khí tượng lớn trên thế giới. Người dùng hiện đại không chỉ đòi hỏi dữ liệu chính xác mà còn yêu cầu các ứng dụng phải có giao diện trực quan, thích ứng cao (Responsive UI/UX) và tốc độ phản hồi tối ưu.

Tuy nhiên, việc triển khai các ứng dụng web tra cứu theo mô hình kiến trúc truyền thống (Monolithic Server hoặc máy chủ ảo VPS) đang vấp phải nhiều rào cản lớn đối với các nhà phát triển độc lập hoặc doanh nghiệp nhỏ, cụ thể bao gồm:

- Thách thức về mặt chi phí hạ tầng: Chủ sở hữu phải duy trì nguồn tài nguyên máy chủ liên tục 24/7 để chờ đợi các yêu cầu tra cứu không liên tục, gây lãng phí và phát sinh chi phí cố định cao ngay cả trong những khung giờ lượng truy cập thấp.
- Khả năng chịu tải kém: Hệ thống truyền thống thường thiếu linh hoạt trong việc tự động mở rộng (auto-scaling), dẫn đến nguy cơ quá tải hoặc sập hệ thống khi nhu cầu tra cứu tăng đột biến trong các hình thái thời tiết cực đoan.
- Nguy cơ lộ lọt thông tin cấu hình (API Key): Nhiều ứng dụng hiện nay thực hiện gọi dữ liệu trực tiếp từ trình duyệt của người dùng (Client-side). Lỗ hổng này khiến các khóa bảo mật dễ dàng bị đánh cắp thông qua các công cụ kiểm tra mã nguồn (F12) hoặc bắt gói tin (Sniffing), đe dọa an toàn thông tin và gây rủi ro cạn kiệt hạn mức dịch vụ.

1.2. Động lực nghiên cứu

Nhận thức được những hạn chế cốt lõi của hạ tầng truyền thống, đề án này hướng tới việc nghiên cứu và ứng dụng Kiến trúc không máy chủ (Serverless Architecture) nhằm tìm kiếm một giải pháp dung hòa giữa hiệu năng, tính kinh tế và độ bảo mật. Động lực lớn nhất của đề tài là việc hiện thực hóa mô hình điện toán đám mây thế hệ mới, giải phóng nhà phát triển khỏi gánh nặng quản lý phần cứng với ba giá trị cốt lõi:

- Về mặt thực tiễn: Chứng minh việc áp dụng các Serverless Functions có thể tối ưu hóa chi phí vận hành xuống mức tối thiểu dựa trên cơ chế cấp phát tài nguyên theo nhu cầu thực tế (Pay-as-you-go).
- Về mặt khoa học kỹ thuật: Giải quyết triệt để bài toán bảo mật API bằng cách thiết lập một lớp Backend trung gian (Middleware) biệt lập trên môi trường đám mây đóng vai trò như một Proxy an toàn để che giấu các khóa cấu hình bí mật.
- Về mặt trải nghiệm: Ứng dụng mạng lưới phân phối biên (Edge Network) toàn cầu để mang lại một hệ thống có độ trễ cực thấp, nâng cao trải nghiệm thị giác và độ tin cậy cho người sử dụng.

1.3. Phạm vi đề tài

Để đảm bảo tính tập trung và khả thi trong khuôn khổ một bài nghiên cứu khoa học, phạm vi của đề tài được giới hạn rõ ràng giữa các hạng mục thực hiện và các yếu tố nằm ngoài mục tiêu nghiên cứu.

Các nội dung đề tài tập trung giải quyết:

- Nghiên cứu cơ chế vận hành của điện toán đám mây thông qua hai nền tảng cốt lõi là AWS Amplify và hệ sinh thái Firebase (BaaS) .
- Xây dựng giao diện ứng dụng web có khả năng tương thích cao (Responsive) trên cả máy tính lẫn thiết bị di động.

- Triển khai phân phối mã nguồn trên hạ tầng của AWS Amplify, kết hợp quản lý biến môi trường để bảo mật API Key từ Windy API và OpenWeatherMap. Xây dựng tính năng định danh và lưu trữ lịch sử qua Firebase Authentication & Firestore.
- Tích hợp dữ liệu thời gian thực thông qua API thời tiết từ các nhà cung cấp uy tín theo tên thành phố và tọa độ định vị của người dùng.

Các nội dung nằm ngoài phạm vi nghiên cứu:

- Đề tài giới hạn không đi sâu vào việc xây dựng cơ sở dữ liệu riêng để lưu trữ dữ liệu lịch sử thời tiết (chỉ xử lý dữ liệu thời gian thực).
- Không phát triển các ứng dụng di động độc lập (Native App như iOS/Android) mà chỉ tập trung tối ưu hiển thị trên trình duyệt di động.

Không nghiên cứu chuyên sâu về các thuật toán dự báo thuộc ngành khí tượng thủy văn.

1.4. Đối tượng nghiên cứu và triển khai

Nội dung nghiên cứu của đề tài được phân định rõ ràng giữa nền tảng lý thuyết và mô hình triển khai thực nghiệm kỹ thuật:

- Đối tượng nghiên cứu (Lý thuyết): Các lý thuyết nền tảng về kiến trúc không máy chủ (Serverless), mô hình tính toán dạng hàm (Function-as-a-Service - FaaS); các giải pháp quản lý, bảo mật API trên Cloud; và các tiêu chuẩn thiết kế UI/UX hiện đại cho ứng dụng hiển thị dữ liệu trực quan.
- Đối tượng triển khai (Kỹ thuật): * Ứng dụng Web (Client-side): Được xây dựng bằng ngôn ngữ lập trình front-end hiện đại và triển khai trên hạ tầng đám mây AWS Amplify.
 - Hệ thống xử lý trung gian bảo mật được thiết lập thông qua biến môi trường của nền tảng này.
 - Cấu trúc dữ liệu: Các chuỗi dữ liệu định dạng JSON nhận về từ các API bên thứ ba để tiến hành tiếp nhận, bóc tách và tinh gọn.

1.5. Kết quả dự kiến

Sau khi hoàn thành các mục tiêu nghiên cứu và thực nghiệm, đề tài kỳ vọng mang lại một số kết quả mang tính ứng dụng cao:

- Sản phẩm công nghệ: Một ứng dụng web tra cứu thời tiết trực tuyến hoàn chỉnh, vận hành ổn định và được phân phối công khai trên Internet. Hệ thống chứng minh được năng lực bảo mật thông qua việc che giấu hoàn toàn API Key trước các hành vi dịch ngược từ phía Client.
- Hệ thống tính năng: Ứng dụng cung cấp khả năng tìm kiếm thông tin thời tiết chi tiết theo thành phố hoặc tự động định vị qua GPS/IP một cách chính xác. Giao diện người dùng đạt được tính trực quan cao, tự động chuyển đổi linh hoạt (Theme, hình nền, biểu tượng) theo trạng thái thời tiết và mốc thời gian thực tế.

CHƯƠNG II: CƠ SỞ LÝ THUYẾT

2.1. Các định nghĩa và khái niệm cơ bản

Để làm nền tảng cho việc thiết kế và triển khai hệ thống tra cứu thời tiết trực tuyến, việc định nghĩa và làm rõ các khái niệm, thuật ngữ công nghệ cốt lõi là điều vô cùng cần thiết. Phần này tập trung khái quát các nền tảng và mô hình kiến trúc then chốt được áp dụng trong toàn bộ đề tài.

2.1.1. Kiến trúc không máy chủ (Serverless Architecture)

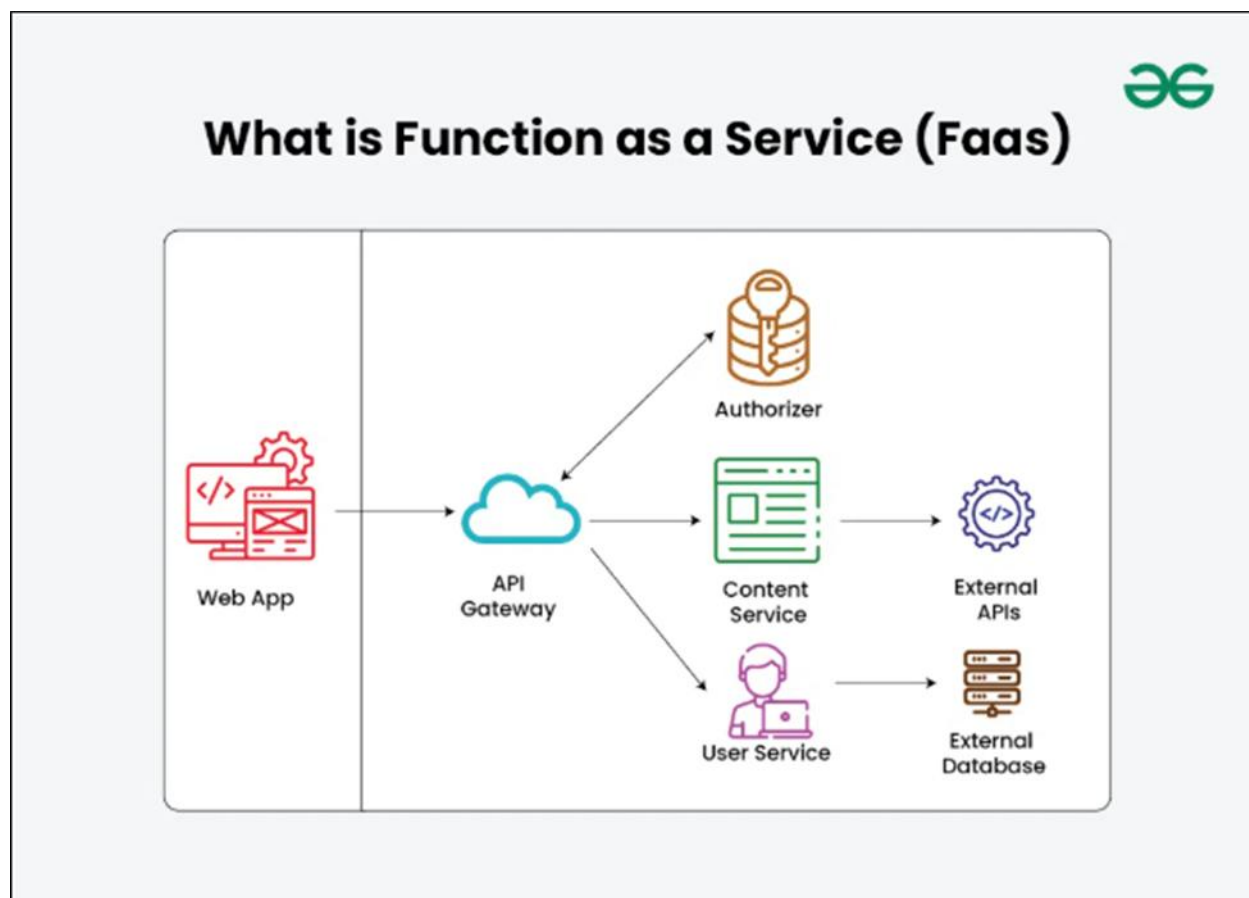


Hình 1 : ServerLess Architecture

Định nghĩa: Serverless (Kiến trúc không máy chủ) là một mô hình điện toán đám mây trong đó nhà phát triển ứng dụng không cần phải quản lý, cấu hình hoặc duy trì hạ tầng máy chủ vật lý hay máy chủ ảo (VPS).

Mặc dù tên gọi là "không máy chủ", thực tế hệ thống vẫn vận hành trên các máy chủ của nhà cung cấp dịch vụ đám mây (Cloud Providers). Tuy nhiên, toàn bộ trách nhiệm về quản lý phần cứng, hệ điều hành, và lỗi bảo mật và tự động mở rộng quy mô (Auto-scaling) đều do nhà cung cấp đảm nhiệm. Trong mô hình này, tài nguyên máy chủ chỉ được kích hoạt và tính phí khi có mã nguồn thực thi kích hoạt bởi một sự kiện cụ thể (Event-driven), giúp tối ưu hóa chi phí theo nguyên tắc "Pay-as-you-go" (chỉ trả tiền cho những gì thực sự sử dụng).

2.1.2. Mô hình FaaS (Function-as-a-Service)



Hình 2 : FaaS

Định nghĩa: FaaS là một dịch vụ điện toán đám mây thuộc kiến trúc Serverless, cho phép các nhà phát triển triển khai và vận hành các khối mã nguồn độc lập (các hàm - Functions) mà không cần xây dựng toàn bộ một hệ thống Backend công kênh.

Mỗi "Function" trong mô hình FaaS là một đoạn mã ngắn, thực hiện một nhiệm vụ duy nhất (ví dụ: tiếp nhận một HTTP Request, xử lý dữ liệu và trả về kết quả). Các hàm này có đặc tính Stateless (không lưu trạng thái) và Ephemeral (vòng đời ngắn) — chúng chỉ khởi chạy khi có yêu cầu gọi đến, thực thi trong vài mili-giây đến vài phút, sau đó tự động hủy bỏ để giải phóng tài nguyên. Trong đề tài này, FaaS đóng vai trò là nền tảng cốt lõi để xây dựng lớp Backend Middleware.

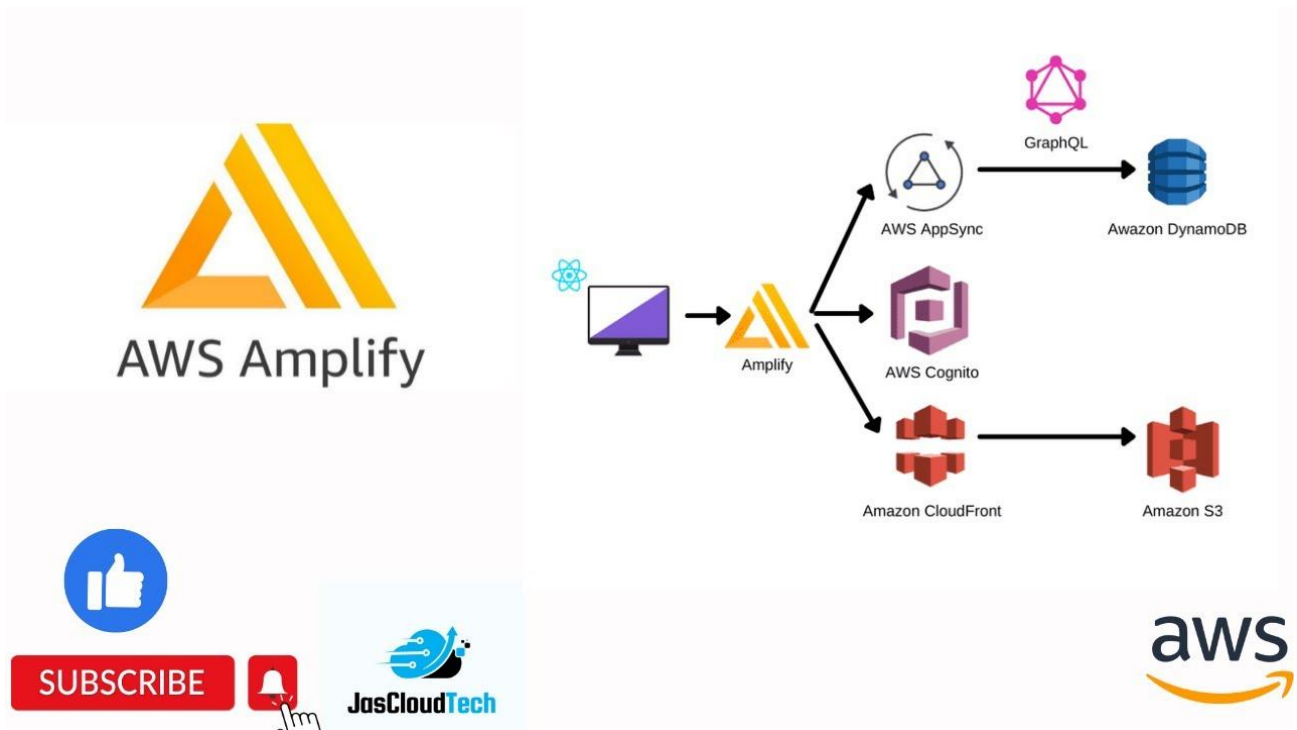
2.1.3. Giao diện lập trình ứng dụng (API) và API Key

Định nghĩa: API (Application Programming Interface) là một tập hợp các quy tắc và giao thức cho phép các phần mềm/hệ thống khác nhau có thể giao tiếp và trao đổi dữ liệu với nhau. API Key là một chuỗi ký tự bí mật đóng vai trò như một mật khẩu để xác thực và cấp quyền cho ứng dụng khi gọi API.

Trong ứng dụng tra cứu thời tiết, hệ thống không tự dự báo thời tiết mà sử dụng API từ các nhà cung cấp dữ liệu khí tượng toàn cầu (ví dụ: Windy.com). Ứng dụng sẽ gửi yêu cầu chứa tên thành phố kèm theo API Key lên máy chủ cung cấp dữ liệu, máy chủ này xác thực mã khóa hợp lệ và trả về chuỗi dữ liệu thời tiết định dạng JSON để ứng dụng bóc tách và hiển thị lên màn hình.

2.1.4. Nền tảng điện toán đám mây AWS Amplify

Định nghĩa: AWS Amplify là một tập hợp các công cụ và dịch vụ đám mây toàn diện do Amazon Web Services cung cấp, được thiết kế chuyên biệt để giúp các nhà phát triển xây dựng và triển khai các ứng dụng Full-stack một cách nhanh chóng và an toàn. Trong phạm vi đề tài, AWS Amplify đóng vai trò là hạ tầng Hosting cốt lõi, cung cấp quy trình CI/CD tự động từ kho lưu trữ mã nguồn (Git). Điểm mạnh của Amplify là khả năng lưu trữ cấu hình bảo mật thông qua Biến môi trường (Environment Variables) ngay trên máy chủ đám mây. Tính năng này giúp che giấu hoàn toàn các khóa API nhạy cảm (API Keys) khỏi trình duyệt của người dùng, triệt tiêu rủi ro bị dịch ngược mã nguồn.

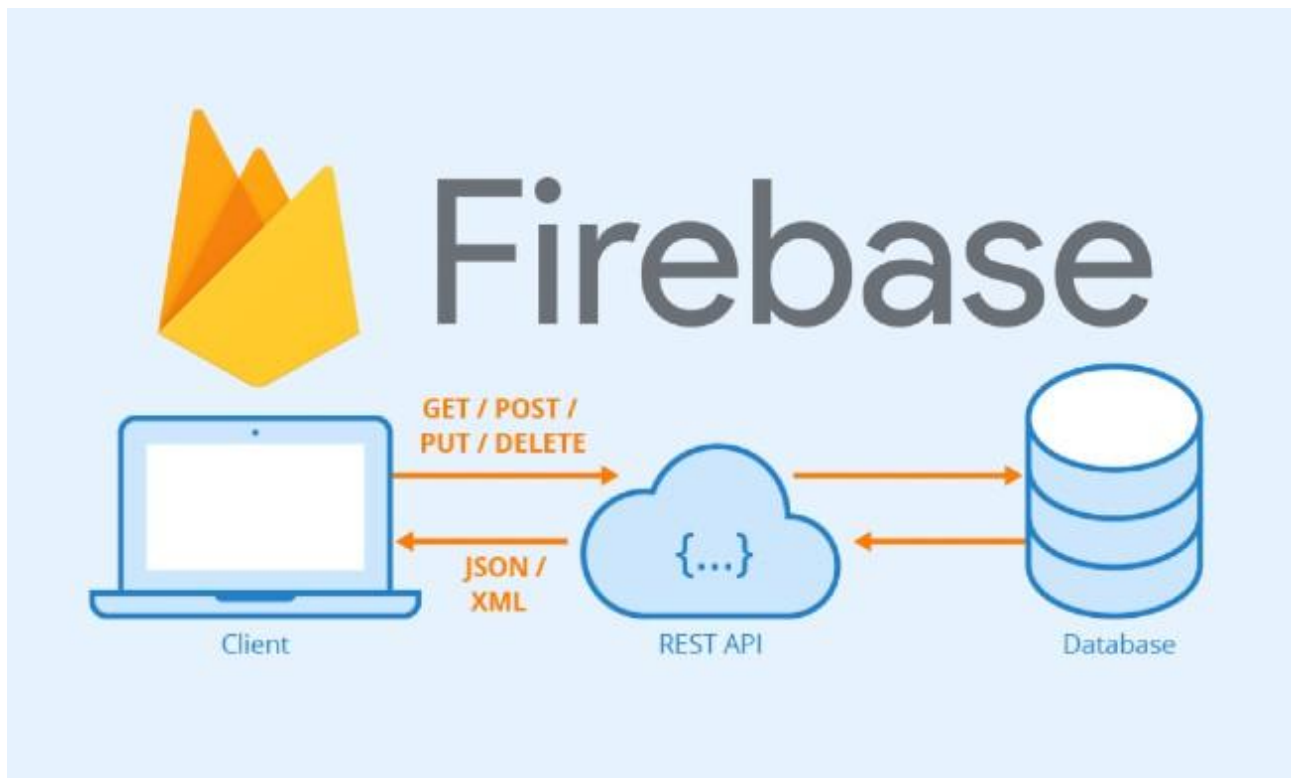


Hình 3 : AWS Amplify

2.1.5. Nền tảng Firebase (Backend-as-a-Service)

Định nghĩa: Firebase là một nền tảng Backend-as-a-Service (BaaS) mạnh mẽ do Google phát triển, cung cấp các dịch vụ hạ tầng phụ trợ sẵn có để nhà phát triển không cần tự xây dựng server. Đề tài áp dụng hai dịch vụ chính của Firebase:

- Firebase Authentication: Cung cấp hệ thống đăng nhập, định danh và cấp quyền bảo mật cho người dùng.
- Cloud Firestore: Cơ sở dữ liệu đám mây dạng NoSQL, cho phép lưu trữ và đồng bộ hóa dữ liệu trực tiếp giữa máy chủ và trình duyệt theo thời gian thực (Real-time). Công cụ này được dùng để xây dựng tính năng "Lịch sử đám mây", lưu lại các thành phố người dùng đã tra cứu.



Hình 4 : Firebase

2.2. Phân loại và so sánh kỹ thuật

Để có một góc nhìn khách quan và đưa ra quyết định lựa chọn công nghệ tối ưu cho hệ thống tra cứu thời tiết, phần này sẽ tiến hành phân tích, so sánh toàn diện giữa mô hình kiến trúc truyền thống và kiến trúc hiện đại, cũng như đặt các nền tảng đám mây phổ biến lên bàn cân kỹ thuật.

2.2.1. So sánh kiến trúc máy chủ Truyền thống (Monolithic/VPS) và kiến trúc không máy chủ (ServerLess)

Trước khi kỷ nguyên điện toán đám mây bùng nổ, việc triển khai một ứng dụng web gọi API ngoài thường phụ thuộc vào các máy chủ ảo (VPS) hoặc máy chủ vật lý truyền thống.

Mô hình truyền thống này yêu cầu một hệ thống Backend hoạt động liên tục (Monolithic Application) chạy trên các framework như Node.js (Express), Python

(Django/FastAPI) để làm nhiệm vụ Proxy. Tuy nhiên, khi đối chiếu với kiến trúc Serverless , mô hình truyền thống bộc lộ nhiều điểm khác biệt lớn về cả hiệu năng lẫn chi phí.

Đặc tính kỹ thuật	Kiến trúc Máy chủ truyền thống (VPS/Dedicated)	Kiến trúc Không máy chủ (Serverless/FaaS)
Quản lý hạ tầng	Nhà phát triển phải tự cấu hình OS, cài đặt môi trường, vá lỗi bảo mật, cài đặt Web Server (Nginx/Apache).	Được trừu tượng hóa hoàn toàn. Nhà cung cấp Cloud tự động quản lý và bảo trì toàn bộ hạ tầng phần cứng/phần mềm bên dưới.
Cơ chế mở rộng (Scaling)	Phải tự cấu hình Load Balancer, thiết lập ngưỡng Auto-scaling phức tạp. Tốc độ mở rộng tính bằng phút.	Tự động mở rộng quy mô theo số lượng Request thực tế (Elastic Scaling). Tốc độ mở rộng tính bằng mili-giây.
Mô hình chi phí	Trả chi phí cố định hàng tháng/hàng năm bất kể máy chủ có tài nguyên rảnh rỗi (Idling) hay không.	Tính phí chính xác theo số lượng yêu cầu (Requests) và thời gian thực thi của mã nguồn (Pay-as-you-go).
Độ sẵn sàng (Availability)	Phụ thuộc vào cấu hình chịu lỗi (Failover) của hệ thống. Nếu một VPS sập, ứng dụng sẽ ngừng hoạt động.	Mặc định có tính sẵn sàng cao (High Availability) nhờ phân tán đa vùng trên hạ tầng đám mây của nhà cung cấp.
Giới hạn thời gian (Timeout)	Không giới hạn thời gian thực thi của một tiến trình xử lý backend (phù hợp tác vụ chạy ngầm rất dài).	Bị giới hạn nghiêm ngặt thời gian thực thi cho mỗi lượt gọi (thường từ 10 giây đến tối đa 15 phút tùy nền tảng).

Hiện tượng Khởi động lạnh	Không xảy ra. Ứng dụng luôn ở trạng thái chạy ngầm và sẵn sàng phản hồi ngay lập tức.	Có thể xảy ra hiện tượng "Cold Start" gây trễ ở lượt gọi đầu tiên sau một khoảng thời gian hàm không hoạt động.
---------------------------	---	---

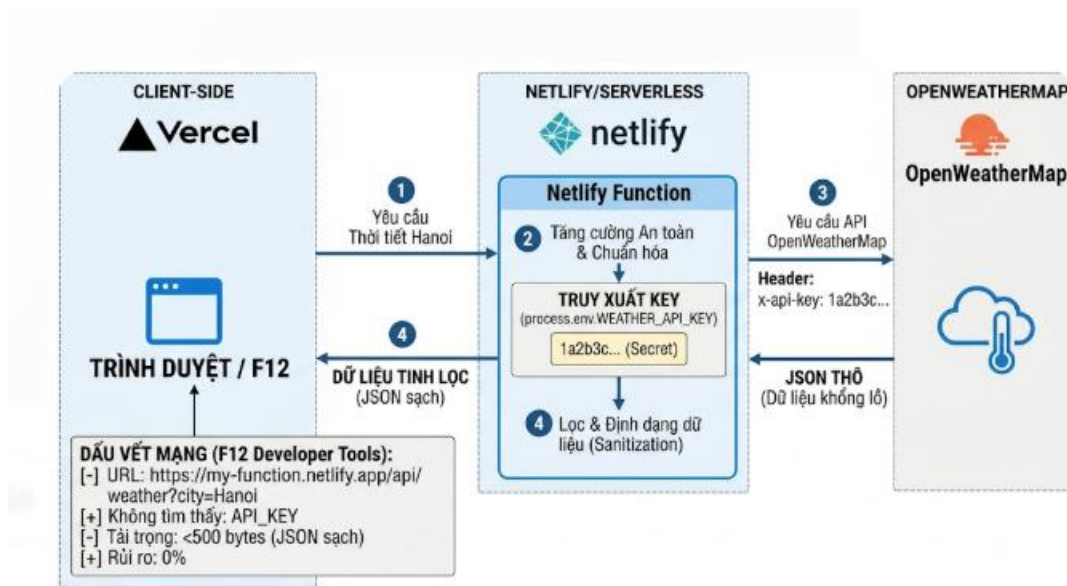
Bảng 2 : So sánh các đặt tính kỹ thuật giữa kiến trúc VPS truyền thống và Serverless

2.2.2. Phân tích ưu điểm của giải pháp AWS Amplify & Firebase .

Sự kết hợp giữa AWS Amplify (quản lý phân phối và bảo mật API) và Firebase (quản lý cơ sở dữ liệu thời gian thực) mang lại một hệ thống phân tán hoàn hảo, giúp tối ưu chi phí hạ tầng, rút ngắn thời gian phát triển và đảm bảo khả năng mở rộng tự động khi lượng truy cập tăng cao.

2.2.3 Nguyên lý luồng dữ liệu bảo mật trong kiến trúc ứng dụng

Để làm rõ nguyên lý lý thuyết về cách hệ thống Serverless bảo vệ thông tin API Key khỏi các cuộc tấn công dịch ngược ở Client-side, sơ đồ dưới đây mô tả luồng di chuyển của dữ liệu từ khi người dùng nhập tên thành phố cho đến khi nhận được kết quả thời tiết hiển thị lên màn hình.



Hình 5 : Minh họa kiến trúc dòng chảy dữ liệu hệ thống

Dựa trên sơ đồ kiến trúc hình 5, nguyên lý vận hành bao gồm 4 bước độc lập:

Bước 1 (Client Request): Trình duyệt của người dùng tại AWS Amplify Frontend gửi một truy vấn HTTPS dạng GET /api/weather?city=Hanoi đến AWS Amplify . Yêu cầu này hoàn toàn không chứa API Key.

Bước 2 (Key Injection): Serverless Function được kích hoạt trong môi trường máy chủ đám mây biệt lập. Tại đây, hàm sẽ tự động truy xuất chuỗi API Key được lưu trữ an toàn trong biến môi trường hệ thống (process.env.WEATHER_API_KEY) và nhúng nó vào tiêu đề (Header) hoặc tham số của yêu cầu mới.

Bước 3 (External Query): Hàm trung gian thực hiện gửi yêu cầu đã có mã khóa bảo mật đến máy chủ API thời tiết quốc tế (WindyMap). Máy chủ đích xác thực Key hợp lệ và trả về gói tin JSON thô chứa lượng dữ liệu khổng lồ.

Bước 4 (Data Sanitization & Response): Hàm Serverless tiến hành lọc bỏ các thông tin thừa, định dạng lại cấu trúc chuỗi dữ liệu gọn nhẹ nhất và trả về cho Client. Người dùng cuối

khi bấm F12 kiểm tra tiến trình mạng chỉ nhìn thấy địa chỉ URL của AWS Amplify chứ hoàn toàn không thể tiếp cận hay phát hiện được API Key của hệ thống.

CHƯƠNG III: PHƯƠNG PHÁP THỰC HIỆN

3.1. Lựa chọn phương pháp tiếp cận

Để giải quyết bài toán cốt lõi về chi phí vận hành máy chủ, khả năng chịu tải và nguy cơ lộ lọt dữ liệu cấu hình, nhóm nghiên cứu quyết định áp dụng các phương pháp sau:

Phương pháp thực nghiệm kỹ thuật: Trực tiếp xây dựng, cấu hình và triển khai mã nguồn trên các môi trường đám mây thực tế (Vercel, Netlify/GCP) thay vì chỉ mô phỏng trên máy chủ cục bộ (Localhost).

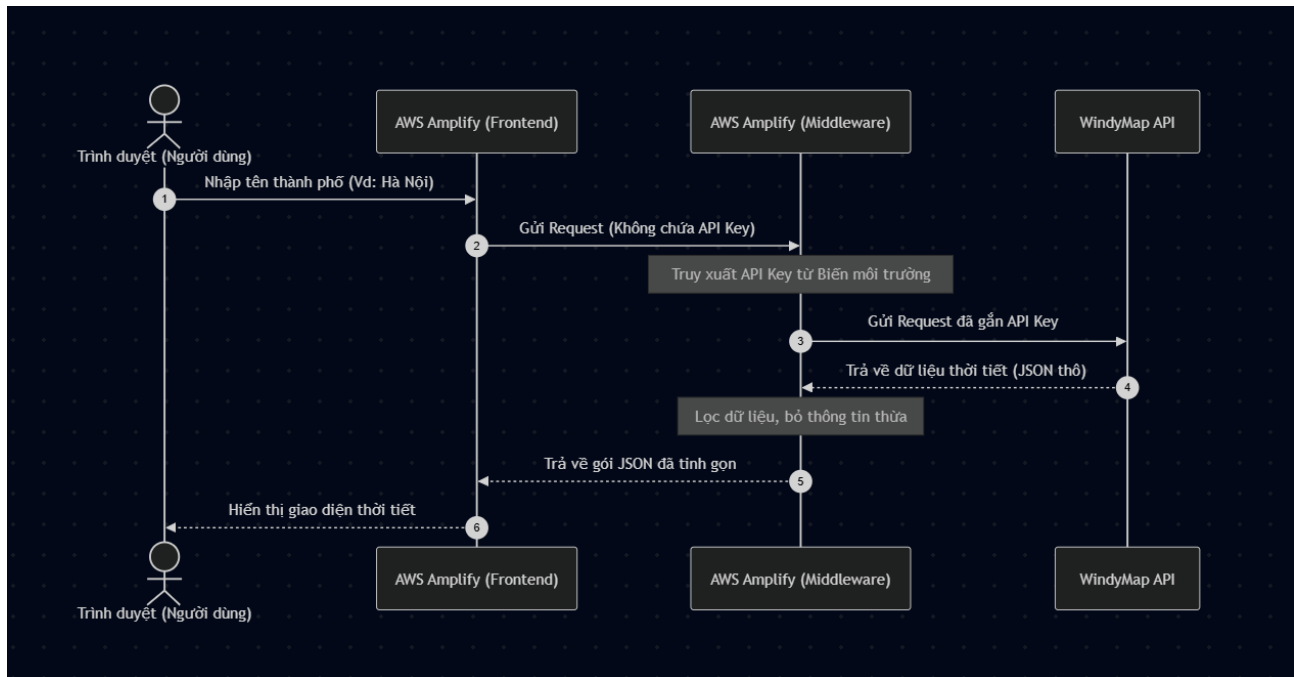
Phương pháp thiết kế hướng dịch vụ vi mô (Microservices/FaaS): Phân tách rõ ràng giữa giao diện hiển thị (Frontend) và logic xử lý trung gian (Backend Middleware) thành các khối độc lập để quản lý linh hoạt.

Lý do lựa chọn công nghệ:

+ AWS Amplify: Được lựa chọn nhờ khả năng phân phối CDN tốc độ cao, hỗ trợ CI/CD tự động và đặc biệt là tính năng quản lý biến môi trường (Environment Variables) giúp giấu kín API Key tuyệt đối an toàn ở phía máy chủ.

+ Firebase (BaaS): Thay thế cho việc phải tự xây dựng cơ sở dữ liệu truyền thống. Firebase cung cấp giải pháp xác thực người dùng và Cloud Firestore giúp đồng bộ hóa dữ liệu lịch sử tìm kiếm ngay lập tức (Real-time).

3.2. Thiết kế hệ thống và luồng xử lý



Hình 6 : Sơ đồ thiết kế hệ thống và luồng xử lý

Hệ thống được thiết kế theo mô hình 3 lớp (Client - Serverless Proxy - External API) với luồng vận hành khép kín nhằm đảm bảo hiệu năng và tính bảo mật:

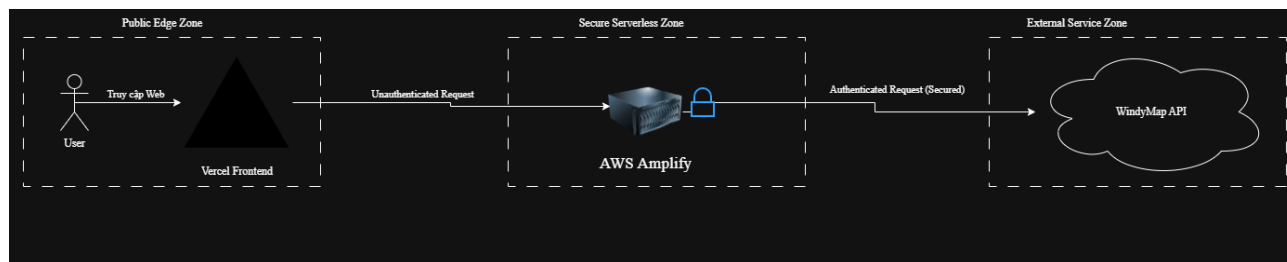
Bước 1: Tiếp nhận truy vấn (Client-side): Người dùng nhập tên thành phố trên giao diện Web. Ứng dụng lập tức gửi một yêu cầu HTTP (ví dụ: GET /api/weather?city=Hanoi) đến điểm cuối (Endpoint) của Serverless Function. Tại bước này, luồng mạng hoàn toàn "sạch" và không chứa khóa bảo mật.

Bước 2: Xử lý trung gian & Bơm khóa bảo mật (Middleware): AWS Amplify tiếp nhận yêu cầu. Tại môi trường máy chủ đám mây biệt lập, đoạn mã sẽ tự động truy xuất API Key từ hệ thống và gắn ngầm vào Header của một yêu cầu mới.

Bước 3: Giao tiếp với nhà cung cấp (External API): Hàm trung gian thay mặt người dùng gửi yêu cầu đã được cấp quyền đến máy chủ cung cấp dữ liệu thời tiết (ví dụ: WindyMap). Máy chủ đích xác nhận Key hợp lệ và trả về một tệp dữ liệu JSON thô.

Bước 4: Tinh giản và Phản hồi dữ liệu: Để tối ưu hóa băng thông tải trang, Serverless Function sẽ tiến hành bóc tách tập JSON thô, loại bỏ các thông số dư thừa và chỉ đóng gói những dữ liệu thiết yếu (Nhiệt độ, độ ẩm, sức gió, icon thời tiết) để trả về cho Frontend hiển thị.

3.3. Sơ đồ kiến trúc mạng



Hình 7 : Sơ đồ kiến trúc mạng

Kiến trúc mạng của ứng dụng được phân lập thành 3 vùng (Zone) cơ bản để tối ưu điều phối lưu lượng và kiểm soát rủi ro truy cập một cách chặt chẽ:

Public Edge Zone (Vùng biên công cộng): Nơi triển khai hạ tầng Frontend (AWS Amplify). Vùng này là điểm tiếp xúc trực tiếp đầu tiên với trình duyệt của người dùng cuối. Nhiệm vụ chính của phân vùng này là chống chịu các đợt truy cập tăng đột biến và đảm bảo tốc độ tải giao diện tĩnh cực nhanh nhờ mạng lưới phân phối toàn cầu.

Secure Serverless Zone (Vùng xử lý an toàn): Chứa các hàm chức năng vận hành trên hạ tầng đám mây (AWS Amplify). Vùng này hoạt động như một "hộp đen" (Blackbox) bảo mật. Mọi tiến trình từ việc tiếp nhận yêu cầu, truy xuất khóa cấu hình (API Key) cho đến bóc tách dữ liệu JSON đều diễn ra ngầm tại đây. Trình duyệt của người dùng từ bên ngoài hoàn toàn không thể can thiệp hay dịch ngược mã nguồn để đánh cắp khóa.

External Service Zone (Vùng dịch vụ ngoại vi): Máy chủ đám mây của tổ chức khí tượng (WindyMap API). Vùng này chịu trách nhiệm tiếp nhận các yêu cầu đã được

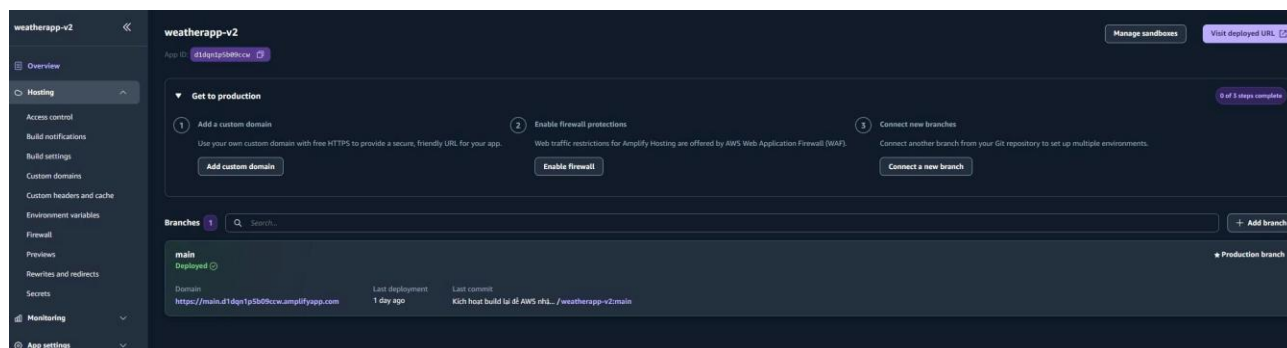
cấp quyền hợp lệ từ Vùng Secure Serverless, sau đó xử lý và cung cấp toàn bộ dữ liệu thời tiết thời gian thực cho ứng dụng.

CHƯƠNG IV: THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1. Triển khai và Demo sản phẩm

4.1.1. Môi trường triển khai thực tế

Hệ thống được triển khai thành công trên môi trường đám mây thực tế thay vì chỉ mô phỏng ở máy chủ cục bộ (Localhost). Mã nguồn giao diện (Frontend) và các kịch bản logic được phân phối trực tiếp qua hạ tầng điện toán đám mây (thể hiện qua đường dẫn URL hoạt động thực tế: `main.d1dqn1p5b09ccw.amplifyapp.com`). Toàn bộ API Key cấu hình cho dịch vụ thời tiết được thiết lập an toàn thông qua biến môi trường (Environment Variables), đóng vai trò như một lớp Middleware để ngăn chặn rủi ro lộ lọt thông tin ở phía trình duyệt người dùng cuối (Client-side).



Hình 8 : Giao diện quản lý dự án (Deploy thành công lên đám mây)

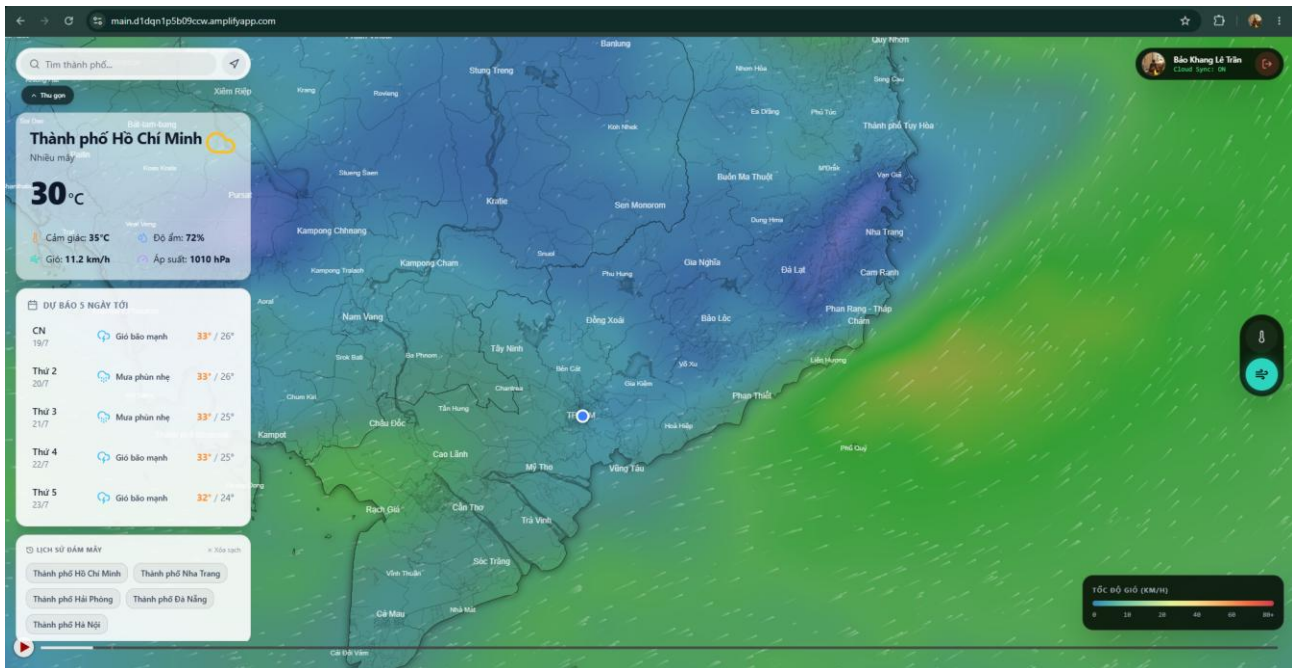
Environment Variables			Manage variables
Environment variables are key/value pairs that contain any constant values your app needs at build time. For instance, database connection details or third-party API keys. Learn more			
Branch :	Variable :	Value :	
All branches	NEXT_PUBLIC_FIREBASE_API_KEY	Alza5y8m5ymzvGz9F8y29SeAD8vHqkcpC8E	
All branches	NEXT_PUBLIC_FIREBASE_APP_ID	1:1049202341358:web:8739f6a3250f257f564aff	
All branches	NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN	cloud-weather-app-37923.firebaseio.com	
All branches	NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID	1049202341358	
All branches	NEXT_PUBLIC_FIREBASE_PROJECT_ID	cloud-weather-app-37923	
All branches	NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET	cloud-weather-app-37923.firebaseiostorage.app	
All branches	NEXT_PUBLIC_WINDY_API_KEY	yfgOJ5+VSUONDIOESBUq08PCu3hyw9R	
			Rows per page: 15

Hình 9 : biến môi trường (Environment variables) trên AWS

4.1.2. Giao diện và Các tính năng cốt lõi

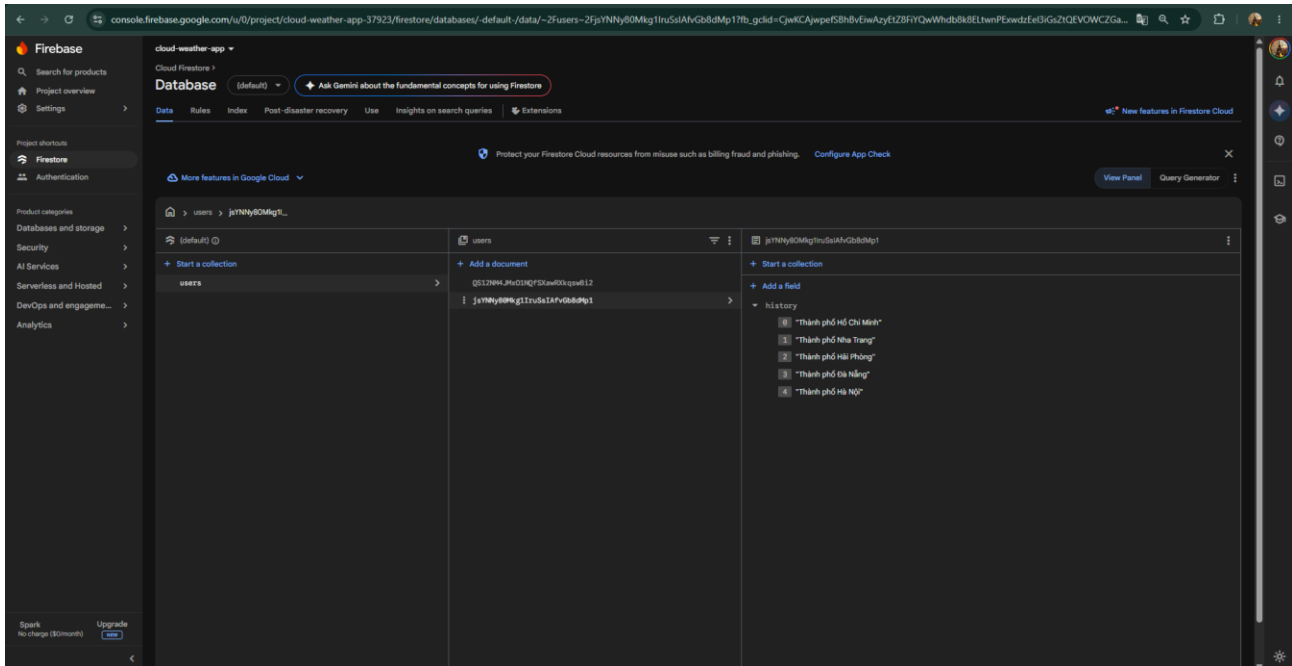
Dựa trên kết quả thực nghiệm, ứng dụng đã hoàn thiện với giao diện trực quan và đầy đủ các tính năng đề ra :

Bản đồ thời tiết động (Windy API) & Tra cứu chi tiết: Khung thông tin nổi (Floating Panel) bên trái cung cấp công cụ tìm kiếm nhanh theo tên thành phố. Khi tra cứu thành công (ví dụ: Thành phố Hồ Chí Minh), hệ thống bóc tách dữ liệu JSON để hiển thị chi tiết các chỉ số hiện tại (Nhiệt độ 30°C, Cảm giác, Độ ẩm, Tốc độ gió, Áp suất) và danh sách cấu trúc dự báo thời tiết cho 5 ngày tiếp theo. Kết hợp với nền tảng bản đồ tương tác Windy cho phép người dùng quan sát trực quan các hình thái khí tượng, chuyển đổi linh hoạt giữa các lớp luồng gió và nhiệt độ.



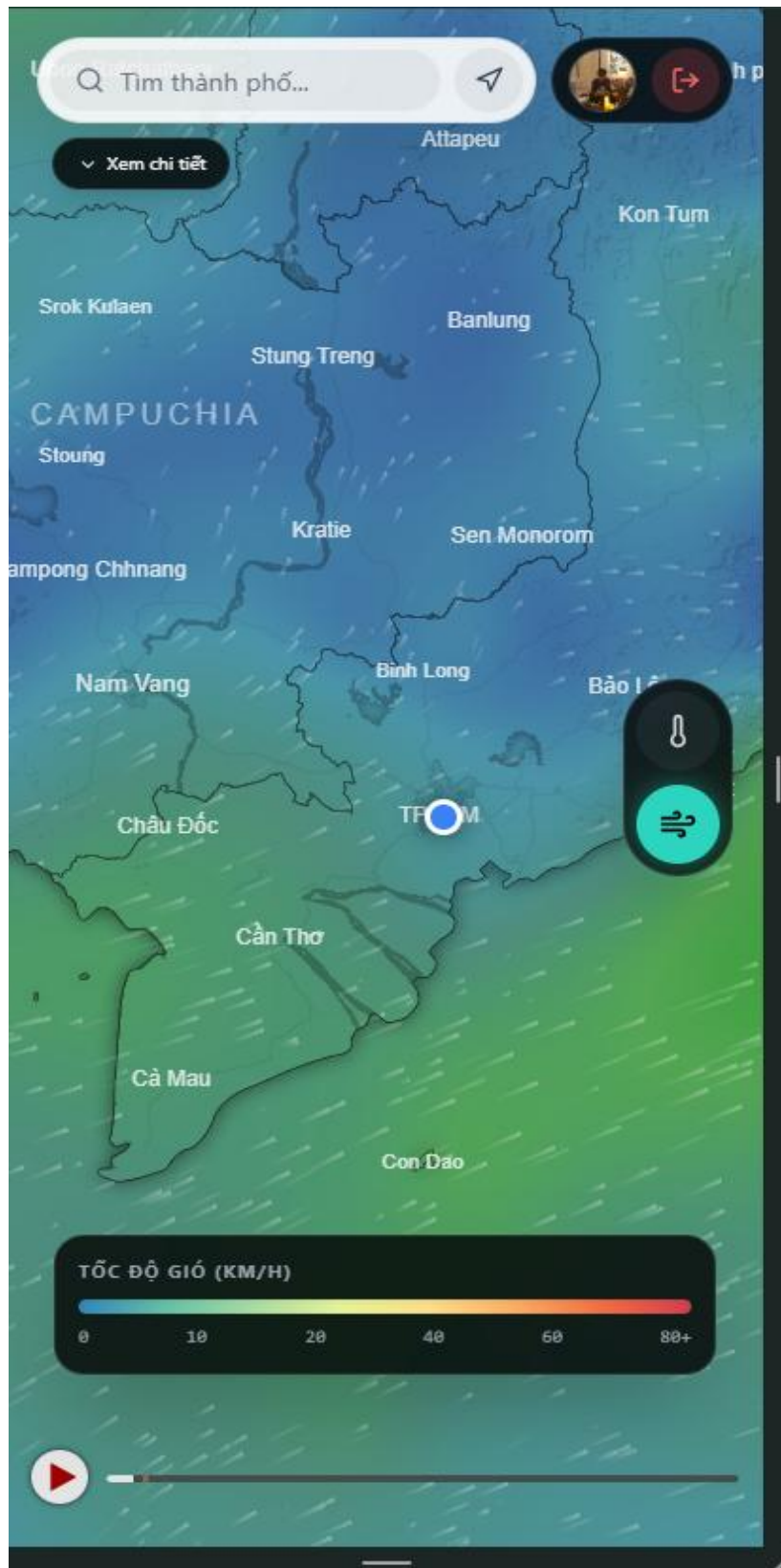
Hình 10 : Giao diện chính của ứng dụng với bản đồ tương tác và bảng tra cứu thời tiết chi tiết

Cá nhân hóa người dùng (Firebase Cloud Sync): Hệ thống tích hợp thành công dịch vụ Firebase Authentication, cho phép định danh người dùng an toàn. Khi người dùng đăng nhập thành công (hiển thị Avatar và tên tài khoản ở góc phải), trạng thái Cloud Sync: ON được kích hoạt. Tính năng "Lịch sử đám mây" ứng dụng Firebase Firestore để tự động lưu lại các địa điểm đã tra cứu (Canada, Úc, Bồ Đào Nha, Bắc Kinh, Hàn Quốc), giúp đồng bộ hóa trải nghiệm cá nhân trên nhiều thiết bị khác nhau.



Hình 11 : Dữ liệu lịch sử tra cứu được lưu trữ và đồng bộ thời gian thực trên Firebase Firestore

Thiết kế đáp ứng (Responsive UI/UX): Ứng dụng được xây dựng theo chuẩn Mobile-First. Giao diện có khả năng tự động nhận diện và co giãn bố cục mượt mà trên mọi kích thước màn hình. Trên các thiết bị di động (Mobile/Tablet), các bảng thông tin nổi (Floating Panel) và thanh công cụ bản đồ được tự động thu gọn hoặc điều chỉnh kích thước hợp lý, đảm bảo người dùng có thể thao tác vuốt, chạm để xem thời tiết một cách thuận tiện nhất mà không bị vỡ giao diện.



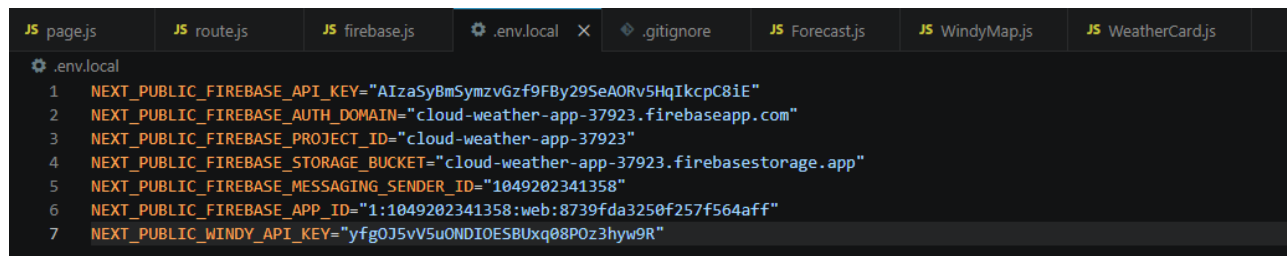
Hình 12 : Giao diện ứng dụng tự động co giãn và tối ưu hiển thị trên thiết bị di động

4.2. Đánh giá kết quả (Evaluation)

4.2.1. Ưu điểm

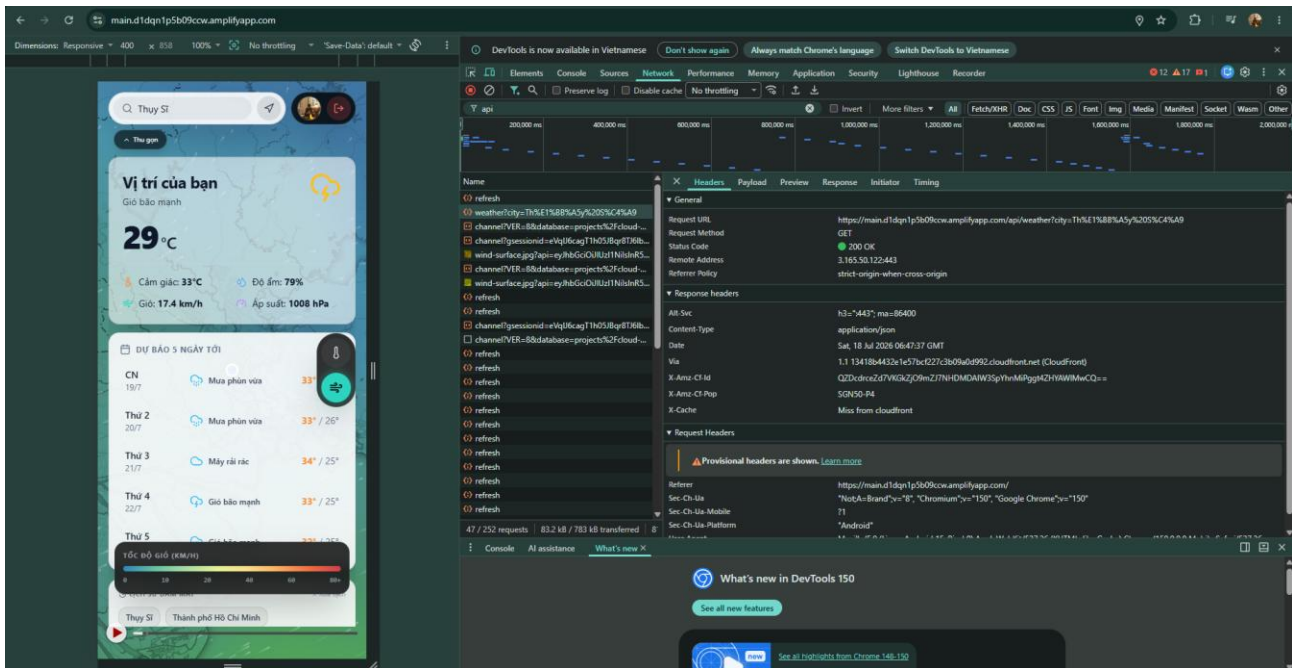
Hiệu năng và Trải nghiệm (UI/UX): Giao diện web được thiết kế hiện đại, đạt chuẩn Responsive thích ứng linh hoạt trên cả PC lẫn Smartphone. Các API thời tiết và nền tảng bản đồ phản hồi với độ trễ rất thấp, thao tác kéo thả và chuyển lớp trên bản đồ diễn ra mượt mà, không gặp hiện tượng nghẽn mạng cục bộ.

Bảo mật tài nguyên xuất sắc: Giải quyết thành công bài toán che giấu khóa cấu hình API Key. Mọi yêu cầu gọi dữ liệu từ nhà cung cấp bên thứ ba đều được xử lý ngầm một cách an toàn, triệt tiêu hoàn toàn rủi ro bị dịch ngược mã nguồn hay đánh cắp khóa thông qua các công cụ kiểm tra mạng (F12 Developer Tools).



```
JS page.js JS route.js JS firebase.js .env.local X .gitignore JS Forecast.js JS WindyMap.js JS WeatherCard.js
.env.local
1 NEXT_PUBLIC_FIREBASE_API_KEY="AIzaSyBmSymzvGzf9FBy29SeAORv5HqIkcpC8iE"
2 NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN="cloud-weather-app-37923.firebaseio.com"
3 NEXT_PUBLIC_FIREBASE_PROJECT_ID="cloud-weather-app-37923"
4 NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET="cloud-weather-app-37923.firebaseio.com"
5 NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID="1049202341358"
6 NEXT_PUBLIC_FIREBASE_APP_ID="1:1049202341358:web:8739fda3250f257f564aff"
7 NEXT_PUBLIC_WINDY_API_KEY="yfg0J5vV5uONDIOESBUXq08POz3hyw9R"
```

Hình 13 : Các API KEY được cất giấu trong .env.local



Hình 14 : Kiểm tra luồng gọi API qua công cụ Developer Tools cho thấy khóa bí mật đã được mã hóa an toàn

Đồng bộ hóa thời gian thực: Cơ sở dữ liệu NoSQL (Firestore) vận hành cực kỳ ổn định, đáp ứng hoàn hảo yêu cầu đọc/ghi dữ liệu lịch sử tìm kiếm ngay lập tức mà không cần tạo mới toàn bộ một hệ thống Backend công kênh.

4.2.2. Hạn chế và Hướng khắc phục

Bên cạnh các kết quả đạt được, ở quy mô thử nghiệm, hệ thống vẫn tồn tại một số hạn chế nhất định về mặt định danh thương hiệu:

Đường dẫn truy cập hiện tại của dự án vẫn đang sử dụng tên miền phụ (Sub-domain) mặc định do nền tảng đám mây cấp phát tự động (có chứa chuỗi ký tự ngẫu nhiên d1dqn1p5b09ccw), chưa tối ưu cho việc ghi nhớ và tra cứu phổ thông.

Hướng khắc phục: Nhóm sẽ tiến hành cấu hình bản ghi DNS (CNAME) và trỏ dự án về một tên miền tùy chỉnh (Custom Domain) mang tính thương hiệu riêng trong bước hoàn thiện cuối cùng, kết hợp xác thực lại luồng bảo mật của API trên tên miền mới để đưa website vào vận hành chính thức.

CHƯƠNG V: KẾT LUẬN

5.1. Kết quả đạt được

Sau quá trình nghiên cứu và thực nghiệm, đề tài đã hoàn thành xuất sắc các mục tiêu đề ra ban đầu, cụ thể:

Triển khai thành công hạ tầng đám mây: Đưa ứng dụng web tra cứu thời tiết vào vận hành thực tế trên nền tảng AWS Amplify với quy trình CI/CD tự động, giúp tối ưu chi phí và đảm bảo tính sẵn sàng cao.

Bảo mật tài nguyên tuyệt đối: Xây dựng thành công lớp Middleware bảo mật thông qua cấu hình biến môi trường (Environment Variables) của AWS Amplify, che giấu hoàn toàn API Key khỏi nguy cơ bị đánh cắp từ phía trình duyệt người dùng (Client-side).

Trải nghiệm UI/UX linh hoạt (Responsive): Ứng dụng được thiết kế theo chuẩn Mobile-First, tự động co giãn bố cục mượt mà, giúp người dùng thao tác tra cứu dễ dàng trên mọi thiết bị (máy tính, điện thoại, máy tính bảng).

Tích hợp dữ liệu và Bản đồ động: Kết nối ổn định với các dịch vụ bên thứ ba như WindyMap API, mang lại trải nghiệm quan sát thời tiết trực quan thông qua các lớp dữ liệu luồng gió và nhiệt độ thời gian thực.

Đồng bộ hóa dữ liệu thời gian thực: Áp dụng hiệu quả hệ sinh thái Backend-as-a-Service của Firebase (Authentication và Cloud Firestore) để triển khai tính năng đăng nhập và lưu trữ "Lịch sử đám mây" mà không cần tự xây dựng máy chủ cơ sở dữ liệu công kênh.

5.2. Hướng phát triển trong tương lai

Dù hệ thống đã vận hành ổn định ở quy mô thử nghiệm, nhóm nghiên cứu vẫn đề xuất một số hướng phát triển để hoàn thiện sản phẩm:

Định danh thương hiệu: Tiến hành cấu hình bản ghi DNS (CNAME) để trỏ dự án về một tên miền tùy chỉnh (Custom Domain), thay thế cho tên miền phụ (Sub-domain) mặc định do hệ thống cấp phát nhằm tối ưu cho việc tra cứu phổ thông.

Mở rộng tính năng: Nghiên cứu tích hợp thêm các API cảnh báo hình thái thời tiết cực đoan (bão, lũ) thông qua SMS hoặc Email cho người dùng đã đăng ký tài khoản.

Tối ưu hiệu suất: Cấu hình sâu hơn các giải pháp bộ nhớ đệm (Caching) ở vùng biên để tiếp tục giảm độ trễ truy vấn dữ liệu khi lượng người truy cập tăng vọt trong cùng một thời điểm.

5.3. Bảng phân công công việc

Thành viên nhóm	MSSV	Vai trò & Hạng mục công việc đảm nhiệm
Bùi Thủy Ngọc Duyên	22DH483978	Thiết kế Frontend cơ bản Lập trình giao diện tĩnh của ứng dụng Web. Thiết kế bố cục (Layout) ban đầu cho các khối thông tin.
Lê Trần Bảo Khang	23DH111490	Lập trình Logic & Tối ưu UI/UX Tích hợp API (WindyMap) và cấu hình dịch vụ Backend (Firebase). Lập trình tính năng cốt lõi (đọc dữ liệu Nhiệt độ, Tốc độ gió, vẽ Thang màu). Xây dựng luồng Cá nhân hóa (Đăng nhập và Lưu trữ dữ liệu lịch sử từng người dùng qua Firestore). Tối ưu hóa thiết kế đáp ứng (Responsive) trên đa thiết bị di động.
Huỳnh Hữu Nghiệp	21DH113908	Triển khai Cloud & Bảo mật (DevOps) Quản trị mã nguồn dự án trên nền tảng GitHub. Xây dựng luồng bảo mật Middleware: Mã hóa và ẩn toàn bộ 7 API Keys cấu hình hệ thống thông qua Biến môi trường.

		Triển khai (Deploy) dự án lên hạ tầng đám mây AWS Amplify.
--	--	--

Bảng 3 : Bảng phân công công việc

5.4. Tài liệu tham khảo

- (1) <https://csrc.nist.gov/publications/detail/sp/800-137/final> , NIST SP 800-137 Information Security Continuous Monitoring (ISCM) for Federal Information Systems and Organizations , 11/03/2026
- (2) <https://csrc.nist.gov/publications/detail/sp/800-61/rev-2/final> , NIST SP 800-61 Rev. 2 Computer Security Incident Handling Guide , 11/03/2026
- (3) <https://owasp.org/API-Security/> , OWASP, "OWASP API Security Top 10 2023", Open Worldwide Application Security Project , 26/06/2026
- (4) <https://www.ibm.com/topics/serverless> , IBM Cloud Education, "Serverless Computing", IBM Cloud Learn Hub , 26/06/2026
- (5) <https://api.windy.com/> , Windy , " Use data trusted by millions ", Windy.com , 18/07/2026
- (6) https://firebase.google.com/docs/firestore/using-console?utm_source=google&utm_medium=cpc&utm_campaign=Cloud-SS-DR-Firebase-FY26-global-gsem-1713590&utm_content=text-ad&utm_term=KW_firebase%20console&gclid=CjwKCAjwyOzSBhBTEiwAmxvJ-oiltqQU7VvUfjk1KRRt08Z1eDW0PjIP6cu_ZLwBxO3usUeKblR03xoCiskQAvD_BwE&hl=vi , Firebase Firestore , Firebase Documentation , 18/07/2026